

```

rus = RandomUnderSampler(random_state=42)
X_Res, y_res = rus.fit_resample(X, y)
print('Resampled dataset shape %s' % Counter(y_res))
Resampled dataset shape Counter({0: 106, 1: 106})

# scatter plot of examples by class label
for label, _ in Counter(y_res).items():
    row_ix = where(y_res == label)[0]
    pyplot.scatter(X_Res[row_ix, 0], X_Res[row_ix, 1],
        label=str(label))
pyplot.legend()
pyplot.show()

```

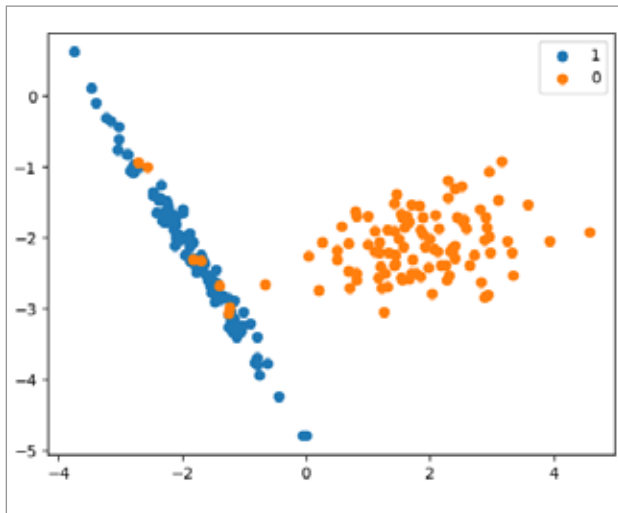


Figure 3: Scatter plot of examples by class label downsampled post using RandomUnderSampler

Cluster centroid

- The centroid algorithm uses the K-Means method to replace the samples under the majority class.
- This method handles the imbalanced datasets and involves undersampling the majority class by replacing a group of majority samples with the cluster centroid of a K-Means algorithm. This newly generated set is created using the centroids of the K-Means method instead of the original samples. By doing this, the majority class(es) are transformed while the minority class remains unchanged.
- The logic is:
 - Choose random data from the majority class.
 - Calculate the Euclidean distance between the random data and its k nearest neighbours.
 - Calculate centroids for clusters and use them to represent the majority class, reducing samples.
 - Repeat the procedure until the desired proportion of minority classes is met.
- The sample code is:

```

from collections import Counter
from sklearn.datasets import make_classification
from sklearn.utils import resample
from imblearn.under_sampling import ClusterCentroids
from sklearn.cluster import MiniBatchKMeans
from matplotlib import pyplot
from numpy import where

X, y = make_classification(n_classes=2, class_sep=2,
    weights=[0.1, 0.9], n_features=4, n_clusters_per_class=1,
    n_samples=1000, random_state=10)

```

```

# summarize class distribution
print('Original dataset shape %s' % Counter(y))
Original dataset shape Counter({1: 894, 0: 106})

```

Now we will carry out downsampling:

```

cc = ClusterCentroids(
    estimator=MiniBatchKMeans(n_init=1, random_state=0), random_
    state=42
)
X_Res, y_res = cc.fit_resample(X, y)
print('Resampled dataset shape %s' % Counter(y_res))
Resampled dataset shape Counter({0: 106, 1: 106})

# scatter plot of examples by class label
for label, _ in Counter(y_res).items():
    row_ix = where(y_res == label)[0]
    pyplot.scatter(X_Res[row_ix, 0], X_Res[row_ix, 1],
        label=str(label))
pyplot.legend()
pyplot.show()

```

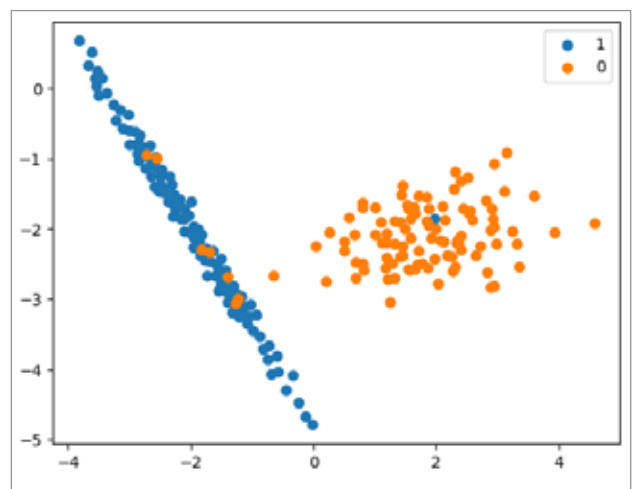


Figure 4: Scatter plot of examples by class label downsampled post using cluster centroids